

OSS4G-IT 2026 OpenLayers Workshop

Pubblicazione su web di dati geospaziali con Openlayers e sviluppo di applicazioni cartografiche con Vuejs Openlayers e Wegue Webgis

Table of contents

1	FOSS4G-IT 2026 OpenLayers Workshop	1
2	FOSS4G-IT 2026 OpenLayers Workshop	3
2.1	Overview	3
2.2	Schedule	3
2.3	Prima di iniziare	3
3	Prima di iniziare	5
3.1	Sviluppare in JavaScript con Node.js	5
3.2	Installazione di Node.js	5
4	Openlayers	7
4.1	Configurazione dell'ambiente di sviluppo	7
4.2	I concetti base di openlayers:	8
4.3	Creare una mappa di base	9
4.4	Creare una mappa con informazioni personalizzate	10
4.5	Modificare lo stile di visualizzazione dei layers vettoriali	12
5	Wegue	15
5.1	Wegue Webgis	15
5.1.1	Installazione	15
5.2	Adattare la configurazione alle proprie esigenze	16
5.2.1	Definire uno stile per un layer vettoriale	18
5.2.2	Aggiungere un layer	18
5.2.3	Moduli aggiuntivi	19
5.3	Localizzazione	22
5.4	Uso di configurazioni alternative	22
5.5	Sistemi di riferimento	23
5.6	QGIS2Wegue	24
5.7	Personalizzazione dell'interfaccia utente	25
5.7.1	Moduli personalizzati	26
5.8	Configurazione finale	29
5.9	Esempio di implementazione di Wegue	29

1 FOSS4G-IT 2026 OpenLayers Workshop

Pubblicare dati geospaziali e realizzare applicazioni cartografiche su web con OpenLayers e WEGUE webgis

2 FOSS4G-IT 2026 OpenLayers Workshop

Pubblicare dati geospaziali e realizzare applicazioni cartografiche su web con OpenLayers e WEGUE webgis



(C)2026 [Enrico Ferreguti](#)

2.1 Overview

Benvenuti al workshop di FOSS4G-IT 2026 dedicato a OpenLayers e WEGUE webgis. In questo laboratorio, imparerete a pubblicare dati geospaziali sul web e a sviluppare applicazioni cartografiche interattive utilizzando le librerie OpenLayers e WEGUE.

2.2 Schedule

14:00 - 16:00	Pubblicazione su web di dati geospaziali con Openlayers
16:00 - 16:15	<i>Coffee Break</i>
16:15 - 18:00	Sviluppo di applicazioni cartografiche con Vuejs Openlayers e Wegue Webgis

2.3 Prima di iniziare

Prima di iniziare il workshop, segui le istruzioni nella pagina [Setup](#) per configurare la tua macchina locale.

3 Prima di iniziare

installazione di Node.js

3.1 Sviluppare in JavaScript con Node.js

Nello sviluppo web tradizionale basato esclusivamente su HTML, ogni pagina viene costruita principalmente come documento statico e l'aggiunta di funzionalità avanzate richiede spesso una gestione manuale di script, dipendenze e organizzazione del codice. Questo approccio può diventare rapidamente difficile da mantenere quando il progetto cresce.

Sviluppare in JavaScript con Node.js è particolarmente vantaggioso per la ricchezza del suo ecosistema di pacchetti. Attraverso **npm (Node Package Manager)**, gli sviluppatori hanno accesso a centinaia di migliaia di librerie già pronte che coprono esigenze comuni come gestione delle interfacce, sviluppo di API, testing, automazione e integrazione con servizi esterni.

Questo approccio consente di ridurre il tempo necessario per implementare funzionalità standard, favorire il riuso del codice e adottare rapidamente strumenti consolidati dalla comunità. L'ampiezza e la maturità dell'ecosistema rendono JavaScript e Node.js una scelta efficace per sviluppare applicazioni moderne in modo rapido e con un elevato livello di flessibilità.

3.2 Installazione di Node.js

1. accedi al sito <https://nodejs.org/en/download/current>
2. installazione con version manager:
 1. seleziona piattaforma (WINDOWS, LINUX, MACOS)
 2. seleziona strumento di installazione (suggerisco nvm per LINUX o MACOS, chocolatey per WINDOWS)
 3. seleziona package manager (suggerisco npm)
 4. apri console (terminal per LINUX o MACOS, powershell per WINDOWS)
 5. esegui i comandi indicati (non servono diritti amministrativi dell'utente)
3. oppure installazione con pacchetto eseguibile (necessario utente con diritti di installazione):
 1. andare alla sezione *get a prebuilt Node.js®*

3 Prima di iniziare

2. Scaricare l'eseguibile binario
3. Eseguire ed installare (verificare diritti amministrativi dell'utente)

Scelta editor

4 Openlayers

Costruire passo dopo passo una semplice applicazione cartografica per il web

4.1 Configurazione dell'ambiente di sviluppo

Prima di procedere con la creazione della nostra applicazione cartografica, è necessario configurare l'ambiente di sviluppo. Assicurati di avere eseguito le attività previste nella sezione “**Prima di iniziare**” per Node.js. A questo punto sarà necessario configurare il nostro ambiente di sviluppo scaricando i files d'impianto [deployment.zip](#) e decomprimendoli nella cartella di lavoro. All'interno della cartella troverai i seguenti files:

<code>data (dir)</code>	Contiene dati geospaziali di esempio
<code>examples (dir)</code>	Contiene esempi tratti da openlayers.org
<code>index.html</code>	Parte Html dell'applicazione
<code>main.js</code>	Parte javascript dell'applicazione
<code>package.json</code>	Contiene i metadati dell'applicazione e le relative dipendenze
<code>package-lock.json</code>	Contiene le dipendenze effettivamente installate
<code>vite.config.js</code>	Contiene la configurazione dell'ambiente di sviluppo

A questo punto bisognerà inizializzare l'ambiente di sviluppo installando tutti i moduli necessari. Apriremo la finestra del terminale nella cartella di lavoro ed andremo a digitare il seguente comando

```
npm install
```

A questo punto potremo far partire il server di sviluppo locale basato su vite.js (che impacchetta i moduli necessari e li consegna al browser) e nodejs

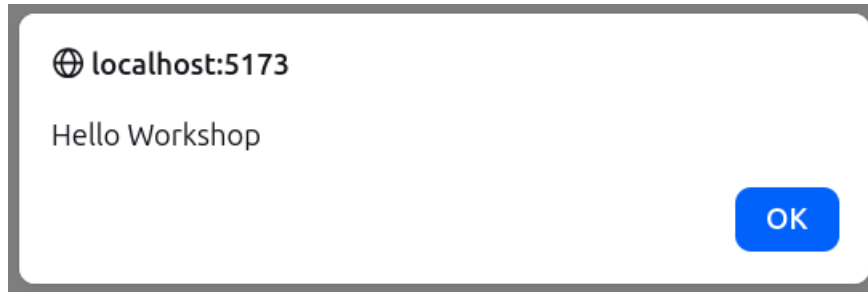
```
npm start
```

Se l'installazione è avvenuta correttamente sulla console appariranno i seguenti messaggi:

```
VITE v3.2.11  ready in 834 ms
```

```
Local:   http://localhost:5173/
Network: use --host to expose
```

Questo significa che saremo in grado di eseguire l'applicazione compilata collegandoci con il browser all'indirizzo <http://localhost:5173/> in cui comparirà una finestra di alert:



A questo punto potremo sviluppare la nostra applicazione cartografica modificando opportunamente il file `index.html` ed il file `main.js` con un editor di testo.

4.2 I concetti base di openlayers:

Prima di iniziare a lavorare con OpenLayers è bene capire i concetti chiave di OpenLayers:

Col1	Col2
Map	La <i>map</i> è un componente chiave di OpenLayers. Per un <i>map</i> da visualizzare, sono necessari una <i>view</i> , uno o più <i>layers</i> e un contenitore di destinazione.
View	La <i>view</i> determina come la mappa è renderizzata. È usata per impostare la risoluzione, le coordinate del centro, ecc. È come una camera attraverso il quale si accede al contenuto della mappa.
Layer	<i>Layers</i> possono essere alla mappa in ordine impilato, questo fa sì che, i layers più in basso sono renderizzati prima dei layers più in alto. Layers possono essere sia <i>layers di immagini</i> (images), che <i>layers vettoriali</i> (punti/linee/poligoni), anche frazionati in tiles
Source	Ogni layer ha una <i>source</i> , che conosce come caricare il contenuto del layer. Nel caso di <i>source vettoriali</i> , la sorgente è letta da dati vettoriali usando una classe <i>format</i> (per esempio GeoJSON o KML) e riempie il layer con un numero di <i>features</i> .
Feature	<i>Features</i> rappresentano cose del mondo reale e possono essere renderizzate con differenti <i>geometries</i> (come punti, linee o poligoni) usando un dato <i>style</i> , che determina il suo aspetto (spessore delle linee, colore di riempimento, etc).
Interaction	La famiglia degli oggetti di interazione permette di interagire con l'oggetto mappa permettendo di indicare punti, creare, modificare e disegnare nuove entità etc
Control	La famiglia degli oggetti di interazione permette di interagire con l'oggetto mappa, modificando la visualizzazione, inserendo dei widget in posizioni prefissate sullo schermo, che fungono da interfaccia utente personalizzabile (per esempio i pulsanti di zoom)

Tutti gli oggetti di Openlayers con la spiegazione precisa dei loro metodi, eventi e proprietà sono descritti nelle [API](#)

L'uso dei vari oggetti è dimostrato tramite [esempi](#)

4.3 Creare una mappa di base

Modificare il file index.html con il seguente contenuto:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Workshop OpenLayers - FOSS4G-IT 2026 Trento</title>
    <style>
      @import "node_modules/ol/ol.css";
    </style>
    <style>
      html, body, #map-container {
        margin: 0;
        height: 100%;
        width: 100%;
        font-family: sans-serif;
      }
    </style>
  </head>
  <body>
    <div id="map-container"></div>
    <script src="./main.js" type="module"></script>
  </body>
</html>
```

modificare poi il file main.js con il seguente contenuto:

```
import OSM from 'ol/source/OSM';
import TileLayer from 'ol/layer/Tile';
import {Map, View} from 'ol';
import {fromLonLat} from 'ol/proj';

new Map({
  target: 'map-container',
  layers: [
    new TileLayer({
      source: new OSM(),
    }),
  ],
});
```

```
view: new View({
  center: fromLonLat([0, 0]),
  zoom: 2,
}),
});
```

Osserviamo che:

- La mappa è contenuta in un'elemento che ha l'id "map-container"
- Che lo stile definito nel codice HTML scala l'elemento a tutto schermo
- Che l'oggetto Map è definito al minimo specificando il target con "map-container" (che deve corrispondere a quello definito nella pagina HTML), un set di layers ed una view
- che l'oggetto layer contiene una sorgente dati (source)
- che il centro della mappa è ricavato con la funzione `fromLonLat`. Di default la mappa viene visualizzata nel sistema di riferimento Web-Mercator e le coordinate espresse in latitudine e longitudine devono essere convertite tramite una funzione

Proviamo a:

- Localizzare la vista su Trento città
- Definire una mappa di determinate dimensioni, per esempio 600 x 600 pixel
- Analizzare errori e connessioni di rete con gli strumenti di sviluppo del browser

4.4 Creare una mappa con informazioni personalizzate

- Aggiungiamo le informazioni relative ai confini amministrativi degli stati del mondo (`data/countries.geojson`)

```
import GeoJSON from 'ol/format/GeoJSON';
import Map from 'ol/Map';
import VectorLayer from 'ol/layer/Vector';
import VectorSource from 'ol/source/Vector';
import View from 'ol/View';
import {fromLonLat} from 'ol/proj';

// attenzione definiamo l'oggetto mappa e la assegnamo ad una variable per altri
//  usi successivi
const map = new Map({
  target: 'map-container',
  layers: [
    new VectorLayer({
      source: new VectorSource({
```

```

        format: new GeoJSON(),
        url: './data/countries.json',
    })),
    ]),
    view: new View({
        center: fromLonLat([0, 0]),
        zoom: 2,
    }),
});

```

- uniamo insieme il layer del confini statali e delle mappe di base

```

import OSM from 'ol/source/OSM';
import TileLayer from 'ol/layer/Tile';
import GeoJSON from 'ol/format/GeoJSON';
import Map from 'ol/Map';
import VectorLayer from 'ol/layer/Vector';
import VectorSource from 'ol/source/Vector';
import View from 'ol/View';
import {fromLonLat} from 'ol/proj';

const stati = new VectorLayer({
    source: new VectorSource({
        format: new GeoJSON(),
        url: './data/countries.json',
    }),
})

const osm = new TileLayer({
    source: new OSM(),
})

const map = new Map({
    target: 'map-container',
    layers: [stati, osm],
    view: new View({
        center: fromLonLat([0, 0]),
        zoom: 2,
    }),
})

```

- e se volessimo aggiungere le regioni italiane ... senza dimenticare di aggiungerlo alla lista dei layers dell'oggetto Map

```
const regioni = new VectorLayer({
  source: new VectorSource({
    format: new GeoJSON(),
    url:
      ↪ 'https://raw.githubusercontent.com/openpolis/geojson-italy/refs/heads/master/geojson/li
      ↪ ,
  }),
});
```

- e se vogliamo cambiare la visualizzazione di default basta intervenire sui parametri dell'oggetto View

```
view: new View({
  center: fromLonLat([12.6, 42.3]),
  zoom: 6,
}),
```

4.5 Modificare lo stile di visualizzazione dei layers vettoriali

lo stile è una proprietà dell'oggetto Layer e deve essere definito per mezzo di un oggetto [Style](#) e che può riguardare vari aspetti della visualizzazione come il colore, il riempimento, lo spessore, il tipo di linea etc... per esempio andiamo a definire uno stile diverso da quello di default per il layer delle regioni italiane:

```
import Fill from 'ol/style/Fill.js';
import Stroke from 'ol/style/Stroke.js';
import Style from 'ol/style/Style.js';

const regioni = new VectorLayer({
  source: new VectorSource({
    format: new GeoJSON(),
    url:
      ↪ 'https://raw.githubusercontent.com/openpolis/geojson-italy/refs/heads/master/geojson/li
      ↪ ,
  }),
  style: new Style({
    stroke: new Stroke({
      color: 'orange',
      width: 4,
    }),
    fill: new Fill({
      color: 'rgba(120, 0, 255, 0.25)',
    }),
  }),
});
```

- possiamo definire lo stile di un layer con una *style function* che genera uno stile dinamicamente a seconda della feature e della risoluzione (non dimentichiamoci di mettere il riferimento allo stile in

```
import Text from 'ol/style/Text.js';

function labelStyle(feature, resolution) {
  return new Style({
    stroke: new Stroke({
      color: 'orange',
      width: 4,
    }),
    fill: new Fill({
      color: 'rgba(120, 0, 255, 0.25)',
    }),
    text: new Text({
      text: feature.get("reg_name"),
    })
  })
}
```


5 Wegue

Costruire passo dopo passo una semplice applicazione cartografica per il web

5.1 Wegue Webgis

Wegue (WebGIS con OpenLayers and Vue) combina insieme il framework Vue.js Con Openlayers per realizzare velocemente applicazioni cartografiche. Mette a disposizione numerosi widget e controlli visuali predefiniti realizzati con il framework Vuetify. Funge da template per ridurre il lavoro d'impianto per applicazioni cartografiche basate su browser.

5.1.1 Installazione

E' possibile scaricare l'applicazione dal repository di github <https://github.com/wegue-oss/wegue> direttamente sotto forma di [file .zip](#) oppure clonare il repository con git (usiamo per uniformità la versione 2 di wegue)

```
git clone https://github.com/wegue-oss/wegue
cd wegue
git checkout v2.0.0
```

Una volta clonato o scaricato ed espanso il file zip è necessario aprire il terminale nella cartella di installazione (riferirsi a “[prima di iniziare](#)”) ed installare i pacchetti javascript necessari:

```
npm install
```

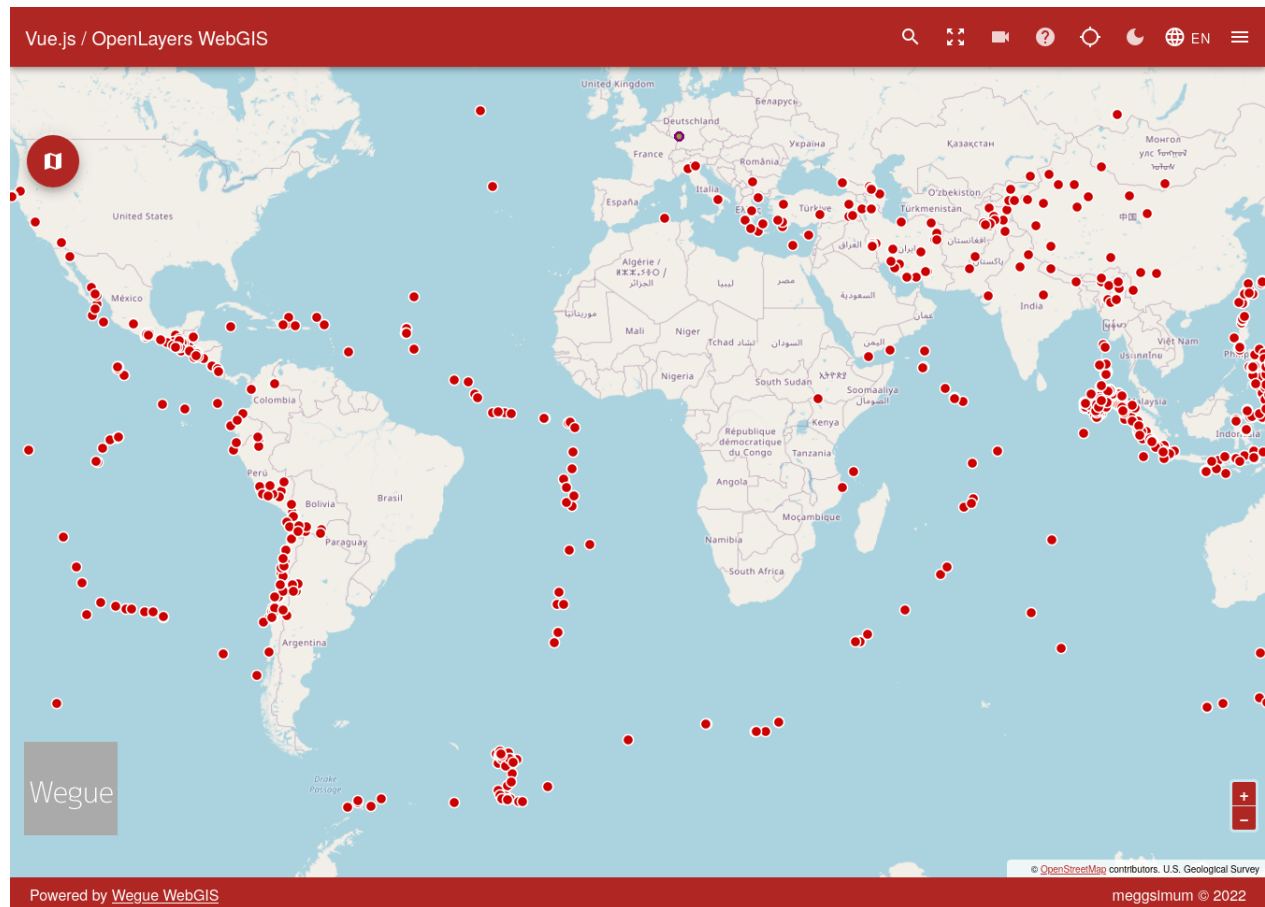
una volta installate le dipendenze bisogna inizializzare l'app Wegue:

```
npm run init:app
```

ed avviare il server di sviluppo che rende disponibile in locale una versione di test dell'applicazione cartografica all'indirizzo <http://localhost:5173/>:

```
npm run dev
```

e dovrebbe apparire la visualizzazione di default dell'applicazione:



alla fine dello sviluppo sarà necessario compilare l'applicazione per ottenere nella cartella `dist` i file da servire in produzione con un server http (apache, nginx etc...)

```
npm run build
```

5.2 Adattare la configurazione alle proprie esigenze

Le componenti configurabili dell'installazione di Wegue sono contenute nella cartella `app` il cui contenuto dovrebbe più o meno corrispondere al seguente:

```
app
  .gitignore
  README.md
  WguApp.vue
  WguAppTemplate.vue

components
  AppFooter.vue
  AppHeader.vue
  AppSidebar.vue
```

```

SampleModule.vue

locales
  de.json
  en.json
  fr.json
  pt.json

public
  static
    app-conf-minimal.json
    app-conf-projected.json
    app-conf-sidebar.json
    app-conf.json

    data
      2012_Earthquakes_Mag5.kml
      shops-dannstadt.geojson

    icon
      circle.svg
      favicon.ico

  styles
    app.css
    vuetify-settings.scss

```

Per cambiare la configurazione basta modificare a nostro piacimento il contenuto di questi files, in particolare la configurazione generale di default è data dal file `public/static/app-conf.json`. Il file controlla tutto quello che può essere visto in Wegue: I vari layers, con le relative configurazioni ed in componenti dell'interfaccia utente. Il file è in formato `json` (Javascript object notation), che è il tipico formato di interscambio di dati strutturati facilmente interpretabile in javascript e *“human readable”*, editabile con un editor di testo

E' particolarmente importante prestare attenzione alla sintassi del file, in quanto anche un minimo errore compromette il funzionamento dell'applicazione.

Il file `app-conf.json` è composto da una serie di direttive, molte delle quali opzionali. La configurazione minima prevede almeno la definizione del centro della mappa e del relativo zoom, e dei layer:

```

{

  "mapZoom": 6,
  "mapCenter": [1384958,5221972],

  "mapLayers": [

    {
      "type": "VECTOR",

```

```

    "lid": "Regioni",
    "url":
      ↪ "https://raw.githubusercontent.com/openpolis/geojson-italy/refs/heads/master/geojson/
      ↪ ,
    "format": "GeoJSON",
    "visible": true
  },

  {
    "type": "OSM",
    "lid": "osm-bg",
    "isBaseLayer": true,
    "visible": true,
    "crossOrigin": "anonymous"
  }
]
}

```

5.2.1 Definire uno stile per un layer vettoriale

Il layer vettoriale usa lo stile di visualizzazione di default, ma è possibile definirne uno di personalizzato aggiungendo la proprietà `style` (prestare attenzione alla virgola di separazione)

```

,
  "style": {
    "strokeColor": "purple",
    "strokeWidth": 3,
    "fillColor": "rgba(155,153,51,0.5)",
    "label": {
      "attribute": "reg_name",
      "outlineColor": "white",
      "outlineWidth": 2,
      "fillColor": "black",
      "offsetX": 0,
      "offsetY": 15,
      "align": "center"
    }
  }

```

5.2.2 Aggiungere un layer

E' possibile aggiungere altri layers, semplicemente inserendo dei nuovi oggetti nidificati nella lista "layers". I layers possono essere di vari tipi: VECTOR, WFS, VECTORTILE, WMS (tiled o image), WMS, XYZ, Arcgis REST:

```

{
  "type": "TILEWMS",
  "lid": "carta geologica",
  "format": "image/png",
  "layers": "GE.CARTAGEOLOGICA",
  "url":
    ↪ "http://wms.pcn.minambiente.it/ogc?map=/ms_ogc/WMS_v1.3/Vettoriali/Carta_geologica.ma
    ↪ ,
  "transparent": true,
  "projection": "EPSG:3857",
  "attribution": "(C)Ministero ambiente",
  "isBaseLayer": false,
  "visible": false,
  "displayInLayerList": true,
  "legend": true,
  "opacityControl": true,
  "crossOrigin": "anonymous"
},

{
  "type": "XYZ",
  "url": "https://mt1.google.com/vt/lyrs=s&x={x}&y={y}&z={z}",
  "lid": "google satellite",
  "isBaseLayer": true,
  "visible": false,
  "crossOrigin": "anonymous"
},

```

5.2.3 Moduli aggiuntivi

E' possibile aggiungere dei moduli aggiuntivi per mezzo della sezione moduli. Andiamo ad aggiungere dopo la sezione layers la sezione moduli con i componenti dedicati alla gestione della visualizzazione dei layers(layerlist), ed all'ottenimento delle informazioni tabellari (infoclick e attributetable)

E' Importante notare che per rispettare la sintassi json va aggiunta una virgola ',' come separatore tra parametri

```

,
  "modules": {
    "wgu-layerlist": {
      "target": "menu",
      "win": "floating",
      "icon": "md:layers",
      "draggable": false
    },
    "wgu-infoclick": {
      "target": "menu",

```

```

        "win": "floating",
        "icon": "md:info",
        "draggable": false,
        "initPos": {
            "left": 8,
            "top": 74
        },
        "showMedia": false
    },
    "wgu-attributetable": {
        "target": "menu",
        "win": "floating",
        "icon": "md:table_chart",
        "syncTableMapSelection": true
    }
}

```

Riassumiamo la configurazione così come definita fino a questo momento

```

{

    "mapZoom": 6,
    "mapCenter": [1384958,5221972],

    "mapLayers": [
        {
            "type": "VECTOR",
            "lid": "Regioni",
            "url":
                ↪ "https://raw.githubusercontent.com/openpolis/geojson-italy/refs/heads/master/geojson/
                ↪ ,
            "format": "GeoJSON",
            "visible": true,
            "style": {
                "strokeColor": "purple",
                "strokeWidth": 3,
                "fillColor": "rgba(155,153,51,0.5)",
                "label": {
                    "attribute": "reg_name",
                    "outlineColor": "white",
                    "outlineWidth": 2,
                    "fillColor": "black",
                    "offsetX": 0,
                    "offsetY": 15,
                    "align": "center"
                }
            }
        }
    ]
}

```

```

},
{
  "type": "TILEWMS",
  "lid": "carta geologica",
  "format": "image/png",
  "layers": "GE.CARTAGEOLOGICA",
  "url":
    ↪ "http://wms.pcn.minambiente.it/ogc?map=/ms_ogc/WMS_v1.3/Vettoriali/Carta_geologica.ma
    ↪ ,
  "transparent": true,
  "projection": "EPSG:3857",
  "attribution": "(C)Ministero ambiente",
  "isBaseLayer": false,
  "visible": false,
  "displayInLayerList": true,
  "legend": true,
  "opacityControl": true,
  "crossOrigin": "anonymous"
},
{
  "type": "XYZ",
  "url": "https://mt1.google.com/vt/lyrs=s&x={x}&y={y}&z={z}",
  "lid": "google satellite",
  "isBaseLayer": true,
  "visible": false,
  "crossOrigin": "anonymous"
},
{
  "type": "OSM",
  "lid": "osm-bg",
  "isBaseLayer": true,
  "visible": true
}
],

"modules": {
  "wgu-layerlist": {
    "target": "menu",
    "win": "floating",
    "icon": "md:layers",
    "draggable": false
  },
  "wgu-infoclick": {
    "target": "menu",
    "win": "floating",
    "icon": "md:info",
    "draggable": false,
    "initPos": {

```

```

        "left": 8,
        "top": 74
    },
    "showMedia": false
},
"wgu-attributetable": {
    "target": "menu",
    "win": "floating",
    "icon": "md:table_chart",
    "syncTableMapSelection": true
}
}
}

```

Altre e più complesse configurazioni si possono ottenere aggiungendo altri moduli, e parametri personalizzati come da istruzioni specifiche: <https://wegue-oss.github.io/wegue/#/wegue-configuration>

5.3 Localizzazione

La webmap può essere facilmente localizzata andando ad aggiungere un file nella cartella locales con la sigla della lingua. Per esempio nel caso si volesse localizzare in italiano si deve creare il file `it.json` (E' opportuno copiare il file `en.json` e sostituire le stringhe target in inglese, *NON LE CHIAVI!*) ed aggiungere degli opportuni parametri di controllo nel file di configurazione:

```

"lang": {
    "supported": {
        "en": "English",
        "it": "Italiano"
    },
    "fallback": "it"
},

```

5.4 Uso di configurazioni alternative

L'applicazione consente di lavorare con differenti configurazioni. Si noti infatti che nella cartella `public/static` sono presenti diversi files che iniziano per `app_conf` e terminano per `.json`. In questo caso è possibile richiamare le configurazioni alternative aggiungendo il parametro `appCtx=[stringa tra app_conf- e .json]`. Per esempio:

File di configurazione in app/public/static	url
app-conf.json	http://localhost:5173/index.html
app-conf-minimal.json	http://localhost:5173/index.html?appCtx=minimal

File di configurazione in app/public/static	url
app-conf-sidebar.json	http://localhost:5173/index.html?appCtx=sidebar
app-conf-projected.json	http://localhost:5173/index.html?appCtx=projected

5.5 Sistemi di riferimento

L'applicazione gestisce nativamente le proiezioni WGS84 (EPSG:4326) e Web Mercator (EPSG:3857), in particolare la finestra di mappa, come in openlayers usa come proiezione predefinita il Web Mercator, quindi eventuali coordinate devono essere espresse in quel sistema di riferimento. E consentito usare proiezioni differenti dichiarandole nella sezione **"projectionDefs"**

```
"projectionDefs": [
  [
    "EPSG:3003",
    "+proj=tmerc +lat_0=0 +lon_0=9 +k=0.9996 +x_0=1500000 +y_0=0 +ellps=intl
    ↪ +towgs84=-104.1,-49.1,-9.9,0.971,-2.917,0.714,-11.68 +units=m +no_defs"
  ],
  [
    "EPSG:32632",
    "+proj=utm +zone=32 +datum=WGS84 +units=m +no_defs"
  ],
  [
    "EPSG:25832",
    "+proj=utm +zone=32 +ellps=GRS80 +towgs84=128.8,17.85,0,0,0,0,0 +units=m
    ↪ +no_defs"
  ]
],
```

Deve essere assegnata la proiezione della finestra di mappa nella sezione **"mapProjection"**

```
"mapProjection": {
  "code": "EPSG:3003",
  "units": "m",
  "extent": [1716014, 5023919, 1737137, 5038662]
},
```

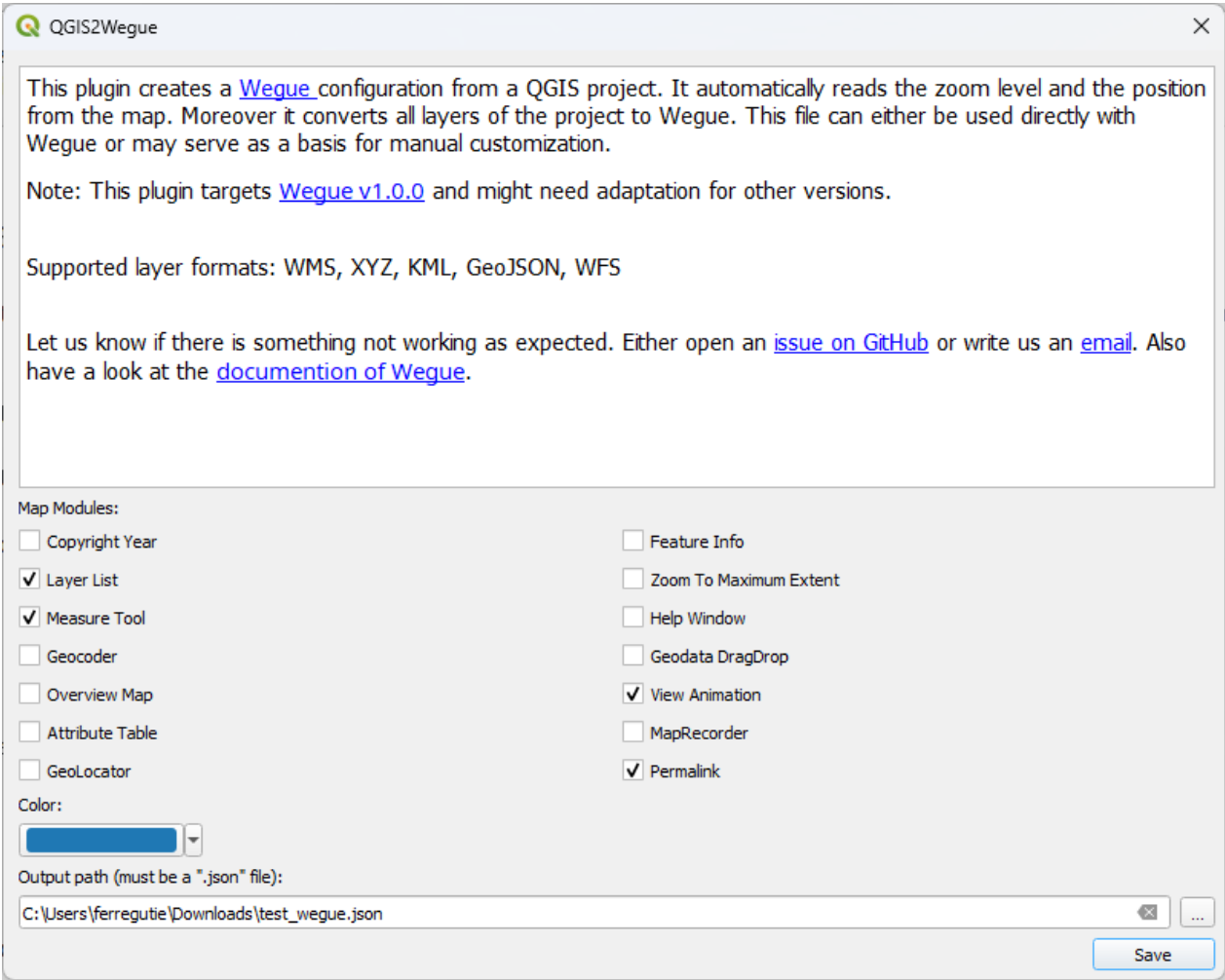
E' dopo necessario assegnare la corretta proiezione ai layer usando la proprietà **"projection"** con la sigla della proiezione definita nella sezione **"projectionDefs"** Qualora non è assegnata la proiezione, viene automaticamente usata la proiezione della mappa stabilita nella sezione **"mapProjection"**

5.6 QGIS2Wegue

Per semplificare la compilazione del file di configurazione esiste un plugin di QGIS scaricabile dal repository ufficiale che si chiama QGIS2Wegue.



Una volta installato il plugin è in grado di generare una configurazione di Wegue a partire dai layers correntemente caricati nel progetto di QGIS importando al contempo la posizione iniziale della mappa ed il relativo livello di zoom. Il Plugin non è molto aggiornato (è del 2022 e genera file di configurazione compatibili con la versione 1.0.0 di Wegue), ne sofisticato, infatti ammette solamente layers del tipo WMS, XYZ, KML, GeoJSON, WFS, non gestisce la ricollocazione delle sorgenti dati su file, non gestisce gli stili e differenti proiezioni, quindi è da considerarsi utile per ottenere iun file di configurazione da adattare e precisare manualmente.



<http://localhost:5173/index.html?appCtx=qgis> render della mappa generata con QGIS2Wegue

5.7 Personalizzazione dell'interfaccia utente

L'applicazione è corredata a una serie di componenti di base la cui presenza nell'interfaccia è governata dalla sezione "modules" nel file `app-conf.json`

Componente	identificativo	descrizione
GeoCoder	wgu-geocoder	Componente per ricercare luoghi geografici per mezzo di servizi web
GeoLocator	wgu-geolocator	Marker personalizzato per il servizio di geo coding
HelpWindow	wgu-helpwin	Modulo per inserire una pagina di spiegazione
InfoClick	wgu-infoclick	Modulo per estrarre informazioni dai layer visibili al click sulla mappa (<i>carotaggio</i>)

Componente	identificativo	descrizione
LayerList	wgu-layerlist	controllo di visibilità e di opacità per i layer correntemente caricati (solo i layer non di base)
MeasureTool	wgu-measuretool	Strumento per misurazioni sulla mappa
ZoomToMaxExtent	wgu-zoomtomaxextent	Bottone per zoomare alla massima estensione degli elementi di mappa
AttributeTable	wgu-attributetable	Tabella degli attributi dei layers vettoriali presenti in mappa
MapRecorder	wgu-maprecorder	Registratore per realizzare storie sulla mappa
LocaleSwitcher	wgu-localeswitcher	Strumento per cambiare il linguaggio di localizzazione in modo interattivo
Map Overlay	wgu-map-overlay	L'overlay è un elemento attaccato alla mappa in una precisa posizione che si sposta al cambiamento della visualizzazione, utile per annotazioni geolocalizzate

5.7.1 Moduli personalizzati

Ma L'aspetto più significativo di Wegue è la possibilità di modificare parti strutturali dell'interfaccia ed aggiungere componenti personalizzati in modo molto semplice.

Prima dell'uso, I componenti devono essere importati e dichiarati nel file `app/WguAppTemplate.vue` nella sezione "components"

Wegue è un'applicazione costruita sul framework [VUE](#) e sulla libreria di componenti visuali [Vuetify](#) ed i componenti sono quindi implementati secondo lo standard del framework che prevede la dichiarazione in un'unico file del template visuale, del codice e dello stile CSS relativo. Wegue aggiunge poi una serie di elementi di help che consentono di usufruire dell'oggetto mappa, e quindi interagire con openlayers e l'EventBus, cioè la logica di emissione e ricezione di segnali che permette di interagire a determinate situazioni, previste dal framework o implementabili dallo sviluppatore.

Per comprendere il meccanismo di implementazione di un componente facciamo riferimento al componente di esempio che sta in `app/components/SampleModule.vue` ed è già precaricato di default nel file di template `app/WguAppTemplate.vue`:

```
<!-- The template contains the HTML of our module -->
<template>
  <!--
```

```

Our module builds upon the 'wgu-module-card' it handles the integration into Wegue
↳ like adding a button in the toolbar
-->
<wgu-module-card
  v-bind="$attrs"
  class="sample-module"
  :icon="icon"
>
  <!--
  Here goes the actual content of the module We use a the component
  ↳ 'v-card-text' from the Vuetify library
  -->
  <v-card-text>
    <!--
    Note the double curly brackets. They contain variables that dynamically
    ↳ change.
    The way how they change is done in the <script> part
    -->
    <b>Zoom:</b> {{ zoom }} <br />
    <b>Center:</b> {{ center }}
  </v-card-text>
</wgu-module-card>
</template>

<script>
// the module card is a the template for a typical Wegue module
import ModuleCard from '@components/modulecore/ModuleCard.vue';
// we import a so called "composable" that helps us to interact with the map
import { useMap } from '@composables/Map';
// an OpenLayers helper function to display coordinates
import { toStringXY } from 'ol/coordinate';
// an OpenLayer helper function to transform coordinate reference systems
import { transform } from 'ol/proj';

export default {
  name: 'sample-module',
  inheritAttrs: false,
  components: {
    'wgu-module-card': ModuleCard
  },
  props: {
    icon: { type: String, required: false, default: 'md:star' }
  },
  // make sure to call the 'useMap' composable and return its resulting bindings
  setup () {
    const { map, layers } = useMap();
    return { map, layers };
  },
}

```

```

// here we define variables that are used in the HTML above
data () {
  return {
    zoom: '',
    center: ''
  };
},
watch: {
  map: {
    handler (newMap) {
      if (newMap) {
        this.onMapBound();
      }
    },
    immediate: true
  }
},
methods: {
  /**
   * This function is called once the map is bound to the application
   */
  onMapBound () {
    // the 'useMap' composable provides access to our OpenLayer map
    // via its returned `map` binding. Returning this binding from
    // the `setup` lifecycle hook makes it accessible via 'this.map'
    // here we get the 'view' from the map
    const view = this.map.getView();
    // we call the function to extract zoom and center from the map
    // once it is initially created
    this.extractMapViewProperties(view);
    // to ensure that we react on updates of the map,
    // we need to register a listener
    view.on('change', () => {
      // always when the map view is changing we extract
      // the current zoom and center from it
      this.extractMapViewProperties(view);
    });
  },

  /**
   * We extract the current zoom and center coordinates
   * from the OpenLayers view and store the values
   * to our module's 'zoom' and 'center' variables
   *
   * @param {ol.View} view The OpenLayers view
   */
  extractMapViewProperties (view) {
    this.zoom = Math.round(view.getZoom());
  }
}

```

```

const sourceCrs = view.getProjection();
const targetCrs = 'EPSG:4326';

// we transform the coordinates from the map projection to WGS84 (EPSG:3857)
const centerTargetCrs = transform(view.getCenter(), sourceCrs, targetCrs);

// create a readable string from the coordinates
this.center = toStringXY(
  centerTargetCrs,
  4 // <-- number of digits after comma
);
}
}
};
</script>

<!-- Here we do the styling of our module -->
<style scoped>

/* our module has the class '.sample-module' and we reference it here */
.sample-module.wgu-floating {
  left: auto !important;
  top: 300px !important;
  right: 10px;
}

</style>

```

Nella configurazione di default il componente si attiva cliccando sull'icona a stella e fa comparire una finestra che contiene le informazioni di zoom e le coordinate del centro visibile della mappa, modificando le informazioni in tempo reale

5.8 Configurazione finale

La configurazione finale dell'applicazione può essere scaricata da [qui](#).

5.9 Esempio di implementazione di Wegue

Ho utilizzato Wegue per fornire un'interfaccia per l'estrazione di dati cartografici di base relativi al comune di Padova. L'applicazione si chiama DBTMAN e consente di estrarre porzioni del rilievo in point cloud del territorio comunale, di estrarre informazioni altimetriche dal DTM e dal DSM e di esportare porzioni del DB topografico in formato CAD

<https://urbanistica.comune.padova.it/dbtman/prod/>

